

Лабораторная работа №2

Выполнила: Ковалевская А.М. 020303-АИСа-024

Оптимизация работы с большими данными

Цель: обеспечить сравнительный анализ эффективности различных методов перемножения больших комплексных матриц.

Задание:

Перемножить 2 квадратные матрицы размера 2048x2048 с элементами типа double complex (комплексное число двойной точности)..

Исходные матрицы генерируются в программе (случайным образом либо по определенной формуле) либо считываются из заранее подготовленного файла.

Оценить сложность алгоритма по формуле $c = 2n^3$, где n - размерность матрицы.

Оценить производительность в MFlops, $p = c/t \cdot 10^{-6}$, где t - время в секундах работы алгоритма.

Выполнить 3 варианта перемножения и их анализ и сравнение:

1-й вариант перемножения - по формуле из линейной алгебры.

2-й вариант перемножения - результат работы функции `cblas_zgemm` из библиотеки BLAS (рекомендуемая реализация из Intel MKL)

3-й вариант перемножения - оптимизированный алгоритм по вашему выбору, написанный вами, производительность должна быть не ниже 30% от 2-го варианта

Листинг программ:

1 вариант

```
import numpy as np
import time
def generate_matrix(n):
    return np.random.rand(n, n) + 1j * np.random.rand(n, n)
def naive_matmul(A, B):
    n = A.shape[0]
    C = np.zeros((n, n), dtype=np.complex128)
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i, j] += A[i, k] * B[k, j]
    return C
n = 2048
A = generate_matrix(n)
B = generate_matrix(n)
start = time.time()
C = naive_matmul(A, B)
end = time.time()
t = end - start
c = 2 * n ** 3
mflops = (c / t) * 1e-6
print("\nНаивный алгоритм:")
print(f"Размер матриц: {n} x {n}")
print(f"Затраченное время: {t:.5f} секунд")
print(f"Производительность: {mflops:.5f} MFlops")
print("Ковалевская А.М., 020303-АИСа-024")
```

2 вариант

```
import numpy as np
import time
from scipy.linalg.blas import zgemm
MATRIX_SIZE = 2048
def generate_complex_matrix(size):
    """Генерация случайной комплексной матрицы заданного размера."""
    return np.random.rand(size, size) + 1j * np.random.rand(size, size)
if __name__ == "__main__":
    matrix_a = generate_complex_matrix(MATRIX_SIZE)
    matrix_b = generate_complex_matrix(MATRIX_SIZE)
    start_time = time.perf_counter()
    result_matrix = zgemm(alpha=1.0, a=matrix_a, b=matrix_b)
    end_time = time.perf_counter()
    operations_count = 2 * MATRIX_SIZE**3
    execution_time = end_time - start_time
    performance_mflops = operations_count / execution_time * 1e-6
    print("\nУмножение матриц с использованием BLAS (zgemm):")
    print(f"Размер матриц: {MATRIX_SIZE}x{MATRIX_SIZE}")
    print(f"Затраченное время: {execution_time:.5f} секунд")
    print(f"Производительность: {performance_mflops:.5f} MFlops")
    print("Ковалевская А.М., 020303-АИСа-о24")
```

3 вариант

```
import numpy as np
import time
from multiprocessing import Pool, shared_memory
import ctypes
SIZE = 2048
BLOCK_SIZE = 128
WORKERS = 8
def init_worker(shared_a, shared_b, shape, dtype):
    global A_shmem, B_shmem
    A_shmem = shared_memory.SharedMemory(name=shared_a)
    B_shmem = shared_memory.SharedMemory(name=shared_b)
    global A, B
    A = np.ndarray(shape, dtype=dtype, buffer=A_shmem.buf)
    B = np.ndarray(shape, dtype=dtype, buffer=B_shmem.buf)
def block_multiply(args):
    i_start = args
    n = A.shape[0]
    block = np.zeros((BLOCK_SIZE, n), dtype=np.complex128)
    for k in range(0, n, BLOCK_SIZE):
        a_block = A[i_start:i_start+BLOCK_SIZE, k:k+BLOCK_SIZE]
        for j in range(0, n, BLOCK_SIZE):
            block[:, j:j+BLOCK_SIZE] += a_block @ B[k:k+BLOCK_SIZE, j:j+BLOCK_SIZE]
    return i_start, block
def parallel_matmul_optimized(A, B):
    n = A.shape[0]
    C = np.zeros((n, n), dtype=np.complex128)
    shm_a = shared_memory.SharedMemory(create=True, size=A.nbytes)
    shm_b = shared_memory.SharedMemory(create=True, size=B.nbytes)
    A_shared = np.ndarray(A.shape, dtype=A.dtype, buffer=shm_a.buf)
    B_shared = np.ndarray(B.shape, dtype=B.dtype, buffer=shm_b.buf)
    np.copyto(A_shared, A)
    np.copyto(B_shared, B)
    with Pool(WORKERS, initializer=init_worker,
              initargs=(shm_a.name, shm_b.name, A.shape, A.dtype)) as pool:
        results = pool.map(block_multiply, range(0, n, BLOCK_SIZE))
    for i, block in results:
```

```

        C[i:i+BLOCK_SIZE, :] = block
    shm_a.close()
    shm_b.close()
    shm_a.unlink()
    shm_b.unlink()
    return C
def benchmark():
    np.random.seed(42)
    A = (np.random.rand(SIZE, SIZE) + 1j*np.random.rand(SIZE, SIZE)).astype(np.complex128)
    B = (np.random.rand(SIZE, SIZE) + 1j*np.random.rand(SIZE, SIZE)).astype(np.complex128)
    _ = parallel_matmul_optimized(A[:256, :256], B[:256, :256])
    start = time.perf_counter()
    C = parallel_matmul_optimized(A, B)
    elapsed = time.perf_counter() - start
    ref = A @ B
    assert np.allclose(C, ref, atol=1e-5), "Ошибка валидации"
    flops = 2 * SIZE**3
    perf = flops / elapsed * 1e-6
    print(f"Оптимизированный параллельный алгоритм:")
    print(f"Размер матриц: {SIZE}x{SIZE}")
    print(f"Затраченное время: {elapsed:.5f} сек")
    print(f"Производительность: {perf:.5f} MFlops")
    print("Ковалевская А.М., 020303-АИСа-о24")
if __name__ == "__main__":
    benchmark()

```

Результаты выполнения программ:

```
= RESTART: C:\Users\alena\OneDrive\Desktop\zzz.py
```

```

Наивный алгоритм:
Размер матриц: 2048 x 2048
Затраченное время: 3580.18 секунд
Производительность: 4.80 MFlops
|

```

```
= RESTART: C:\Users\alena\OneDrive\Desktop\ll.py
```

```

Умножение матриц с использованием BLAS (zgemm):
Размер матриц: 2048x2048
Затраченное время: 0.78837 секунд
Производительность: 21791.62958 MFlops
|

```

```
= RESTART: C:\Users\alena\OneDrive\Desktop\kk2.py
```

```

Оптимизированный параллельный алгоритм:
Размер матриц: 2048x2048
Затраченное время: 1.18408 сек
Производительность: 14509.10190 MFlops

```

Вывод: использование специальных библиотек и собственных оптимизаций значительно повышает скорость выполнения матричных операций по сравнению с базовым методом, что важно для обработки больших данных и сложных вычислительных задач.

